SAME DAY SOLUTION PVT. LTD.

Partner Reconciliation API

Integration Guide — Transaction Status & Reconciliation

Partner Reconciliation API — Integration Guide

Automated status & reconciliation for integration partners.

This API lets a partner pull the **exact status of every transaction** they performed through us — without logging into the partner portal or checking transactions one by one.

Two services are covered:

#	Service	What it is	Status source
1	**Pay2New (BBPS-2 Credit Card)**	Bill payments	`partner_wallet_ledger`
2	**Settlement-2 (SHADVAL Pay)**	Bank account transfers	`shadval_settlement`

---

1. Base URL

```
Production : https://app.samedaysolution.in
```

All endpoints are under `/api/partner/...`.

---

2. Authentication (HMAC-SHA256)

Every request is signed. Send these three headers on **every** call:

Header	Value
`x-api-key`	Your public API key
`x-timestamp`	Current Unix time in **milliseconds**
`x-signature`	`HMAC_SHA256(api_secret, body + timestamp)` as lowercase hex

Where `body` is the **canonical JSON string** of the request body:

```
body = JSON.stringify(parsedRequestBody)      // for POST
body = ""                                     // for GET / empty body
signature = HMAC_SHA256(api_secret, body + timestamp)   // hex
```

The server re-parses and re-stringifies your JSON before verifying, so **whitespace / formatting in your raw body does not matter** — only the key/value content does.

Preconditions (ask admin to configure once)

- The **Transaction Status / Reconciliation API** flag must be enabled for your partner account. This is a dedicated master switch for all status/reconciliation endpoints below. If it is off, every status call returns `403 STATUS_API_DISABLED` ("...this option is disabled — please contact support").
- The relevant **service flag** must be enabled for your partner account:
  - Pay2New → **BBPS-2** enabled
  - Settlement-2 → **Settlement-2** enabled
- Your server's **public IP must be whitelisted** (mandatory — no whitelist = blocked).
- `x-timestamp` must be within **5 minutes** of server time (prevents replay). Keep your server clock in sync (NTP).

Signing example (Node.js)

```
const crypto = require('crypto');

function signedHeaders(apiKey, apiSecret, bodyObj) {
  const timestamp = Date.now().toString();
  const body = bodyObj ? JSON.stringify(bodyObj) : '';
  const signature = crypto
    .createHmac('sha256', apiSecret)
    .update(body + timestamp)
    .digest('hex');
  return {
    'Content-Type': 'application/json',
    'x-api-key': apiKey,
    'x-timestamp': timestamp,
    'x-signature': signature,
  };
}

// Usage
const body = { from_date: '2026-10-01', to_date: '2026-10-14', page: 1, page_size: 100 };
const res = await fetch('https://app.samedaysolution.in/api/partner/pay2new/bill/status/bulk', {
  method: 'POST',
  headers: signedHeaders(API_KEY, API_SECRET, body),
  body: JSON.stringify(body),   // MUST be the same object you signed
});
```

> **Important:** the object you `JSON.stringify` for the body **must be the same** one you signed. Don't re-order keys or re-serialize differently between signing and sending.

Signing example (PHP)

```
$timestamp = (string) round(microtime(true) * 1000);
$body = json_encode($payload); // same string you send
$signature = hash_hmac('sha256', $body . $timestamp, $apiSecret);
$headers = [
  'Content-Type: application/json',
  'x-api-key: ' . $apiKey,
  'x-timestamp: ' . $timestamp,
  'x-signature: ' . $signature,
];
```

---

3. Common request contract (bulk endpoints)

Field	Type	Required	Notes
`from_date`	string	yes	`YYYY-MM-DD` or ISO 8601. Date-only snaps to `00:00:00.000Z`.
`to_date`	string	yes	`YYYY-MM-DD` or ISO 8601. Date-only snaps to `23:59:59.999Z`.
`page`	number	no	Default `1`.
`page_size`	number	no	Default `50`, **max** `200`.
`status`	string	no	Filter by status (values differ per service — see below).

Limits: date range ≤ **92 days** per call. Split larger windows.

Every bulk response includes a `pagination` block:

```
"pagination": {
  "page": 1,
  "page_size": 100,
  "total_records": 342,
  "total_pages": 4,
  "has_next": true,
  "has_prev": false
}
```

---

4. Endpoints

4.1 Pay2New — Bulk Bill Payment Status

```
POST /api/partner/pay2new/bill/status/bulk
```

Request

```
{
  "from_date": "2026-10-01",
  "to_date": "2026-10-14",
  "page": 1,
  "page_size": 100,
  "status": "SUCCESS"
}
```

`status` (optional): `SUCCESS` | `FAILED` | `PENDING` | `REFUNDED`.

Response `200`


```
{
  "success": true,
  "service": "pay2new",
  "from_date": "2026-10-01T00:00:00.000Z",
  "to_date": "2026-10-14T23:59:59.999Z",
  "status_filter": "SUCCESS",
  "pagination": { "page": 1, "page_size": 100, "total_records": 342, "total_pages": 4, "has_next": true, "has_prev": fa
  "count": 100,
  "transactions": [
    {
      "request_id": "SDS1760000000000",
      "bill_fetch_ref": "BF-abc123",
      "order_id": "P2N_PAY_998877",
      "status": "SUCCESS",
      "amount": 2500.00,
      "charge": 29.50,
      "operator_reference": "OPREF123456",
      "created_at": "2026-10-03T09:12:44.000Z",
      "updated_at": "2026-10-03T09:13:10.000Z"
    }
  ]
}
```


> **Note on `status` + pagination:** Pay2New status is *derived* (not a single DB column), so the `status` filter is applied **after** fetching the page. `pagination` reflects the **unfiltered** range. For exact filtered counts, either iterate all pages, or omit `status` and filter client-side. (Settlement-2 below filters at the DB level, so its pagination is exact.)

Status	Meaning
`SUCCESS`	Paid; `order_id` present.
`PENDING`	Still being confirmed with the provider.
`FAILED`	Payment failed.
`REFUNDED`	Failed and the amount was returned to your wallet.

4.2 Pay2New — Single Bill Payment Status (live)

```
POST /api/partner/pay2new/bill/status
```

Provide **one** of: `request_id`, `order_id`, `bill_fetch_ref`. Performs an authoritative **upstream re-check** for still-pending rows — use this when you need a *live* status for one transaction.

```
{ "request_id": "SDS1760000000000" }
```

4.3 Settlement-2 — Bulk Account Transfer Status

```
POST /api/partner/settlement/status/bulk
```

Request

```
{
  "from_date": "2026-10-01",
  "to_date": "2026-10-14",
  "page": 1,
  "page_size": 100,
  "status": "SUCCESS"
}
```

`status` (optional): `SUCCESS` | `FAILED` | `PENDING` | `AWAITING_APPROVAL` | `REJECTED` (filtered at DB level — pagination is exact).

Response `200`

```
{
  "success": true,
  "service": "settlement-2",
  "from_date": "2026-10-01T00:00:00.000Z",
  "to_date": "2026-10-14T23:59:59.999Z",
  "status_filter": null,
  "pagination": { "page": 1, "page_size": 100, "total_records": 156, "total_pages": 2, "has_next": true, "has_prev": fa
  "count": 100,
  "transactions": [
    {
      "id": "a1b2c3d4-...",
      "reference_id": "SDSSETTLE123",
      "order_id": "SV_ORDER_55",
      "utr": "UTR20261003XYZ",
      "amount": 10000.00,
      "charges": 5.90,
      "total_debited": 10005.90,
      "mode": "IMPS",
      "status": "SUCCESS",
      "status_message": "Transaction Successful",
      "account_number": "9632587410",
      "ifsc_code": "SBIN0001234",
      "account_holder_name": "TEST USER",
      "created_at": "2026-10-03T10:00:00.000Z",
      "provider_timestamp": "2026-10-03 10:00:05"
    }
  ]
}
```


Status	Meaning
`SUCCESS`	Money transferred; `utr` present.
`PENDING`	In progress at the bank/provider.
`FAILED`	Transfer failed; wallet refunded.
`AWAITING_APPROVAL`	Queued for admin approval (no provider call yet).
`REJECTED`	Rejected by admin; wallet refunded.

4.4 Settlement-2 — Single Account Transfer Status (live)

```
GET /api/partner/settlement/status?reference_id=SDSSETTLE123
GET /api/partner/settlement/status?list=true&limit=50
```

Live re-check for one transfer (refreshes PENDING from the provider). `list=true` returns a quick recent list (max 50) without a date range.

4.5 Transactions — Reconciliation Summary

```
POST /api/partner/transactions/summary
```

Request

```
{ "from_date": "2026-10-01", "to_date": "2026-10-14" }
```

Response `200`

```
{
  "success": true,
  "from_date": "2026-10-01T00:00:00.000Z",
  "to_date": "2026-10-14T23:59:59.999Z",
  "pay2new": {
    "total": 342, "success": 310, "failed": 20, "pending": 8, "refunded": 4,
    "total_amount": 785000.00, "total_charge": 9140.50, "truncated": false
  },
  "settlement": {
    "total": 156, "success": 140, "failed": 10, "pending": 6,
    "awaiting_approval": 0, "rejected": 0,
    "total_amount": 1250000.00, "total_charges": 826.00, "truncated": false
  }
}
```

- Only services **enabled** for the partner appear.
- `total_amount` counts **SUCCESS** transactions only (money that actually moved).
- `truncated: true` means the range exceeded the aggregation cap (50,000 rows) — narrow the window.

---

5. Error format

All errors share one shape:

```
{ "success": false, "error": { "code": "BAD_REQUEST", "message": "from_date and to_date are required (YYYY-MM-DD or ISO
```

HTTP	`code`	Cause
400	`BAD_REQUEST`	Missing/invalid params or bad JSON.
400	`RANGE_TOO_LARGE`	Date range > 92 days.
401	`UNAUTHORIZED`	Bad/missing signature, stale timestamp, IP not whitelisted, inactive key.
403	`STATUS_API_DISABLED`	The Transaction Status / Reconciliation API is not enabled for this partner — contact support.
403	`FORBIDDEN`	Underlying service (BBPS-2 / Settlement-2) not enabled for this partner.
404	`ORDER_NOT_FOUND` / `NOT_FOUND`	(single-txn) no matching transaction.
500	`INTERNAL_ERROR`	Server-side failure.

---

6. Recommended reconciliation flow (automation)

1. **Nightly / hourly pull** — call the two **bulk** endpoints for the window since your last sync (e.g. last 24–48h), paging through all results. Store/update each transaction by `reference_id` / `request_id`.
2. **Resolve PENDING** — for any row still `PENDING` after a few hours, call the matching **single-txn (live)** endpoint to force an authoritative provider re-check.
3. **Dashboard** — call the **summary** endpoint for at-a-glance totals by status.
4. **(Optional) Real-time** — subscribe to our **webhooks** so you're pushed each terminal state as it happens and barely need to poll:
   - `pay2new.cc.status` — Pay2New terminal state (`SUCCESS` / `FAILED` / `REFUNDED`).
   - `settlement.*` — Settlement-2 terminal state. Webhooks are signed with your `webhook_secret` : `X-Sameday-Signature = HMAC_SHA256(webhook_secret, timestamp + "." + rawBody)` with headers `X-Sameday-Timestamp` , `X-Sameday-Delivery` , `X-Sameday-Event` . Ask admin to register your webhook URL(s).

---

7. Using the Postman collection

1. Import `Partner-Reconciliation-API.postman_collection.json` .
2. Set collection (or environment) variables: `base_url` , `api_key` , `api_secret` , and optionally `from_date` / `to_date` .
3. Send any request — the collection's **Pre-request Script signs it automatically** (computes `x-api-key` , `x-timestamp` , `x-signature`).

> If you see `401 UNAUTHORIZED` in Postman, disable **Settings → General → "Send requests via cloud agent"** so your real IP (the whitelisted one) is used.

---

8. Quick reference

Purpose	Method	Path
Bulk bill-payment status	POST	`/api/partner/pay2new/bill/status/bulk`
Single bill-payment status (live)	POST	`/api/partner/pay2new/bill/status`
Bulk account-transfer status	POST	`/api/partner/settlement/status/bulk`
Single account-transfer status (live)	GET	`/api/partner/settlement/status?reference_id=…`
Reconciliation summary	POST	`/api/partner/transactions/summary`